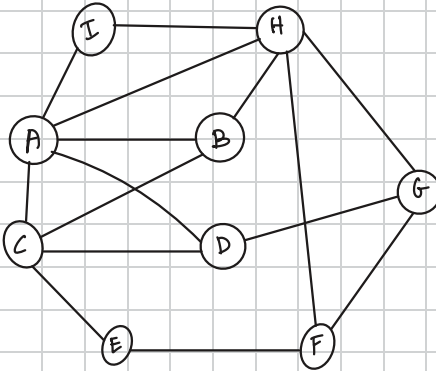


Assignment 6

Task 1:

(a)



A	A	B	C	D	E	F	G	H	I
A	0	1	1	1	0	0	0	1	1
B	1	0	1	0	0	0	0	1	0
C	1	1	0	1	1	0	0	0	0
D	1	0	1	0	0	0	1	0	0
E	0	0	1	0	0	1	0	0	0
F	0	0	0	0	1	0	1	1	0
G	0	0	0	1	0	1	0	1	0
H	1	1	0	0	0	1	1	0	1
I	1	0	0	0	0	0	0	1	0

adjacency matrix A

D	A	B	C	D	E	F	G	H	I
A	5	0	0	0	0	0	0	0	0
B	0	3	0	0	0	0	0	0	0
C	0	0	4	0	0	0	0	0	0
D	0	0	0	3	0	0	0	0	0
E	0	0	0	0	2	0	0	0	0
F	0	0	0	0	0	3	0	0	0
G	0	0	0	0	0	0	3	0	0
H	0	0	0	0	0	0	0	5	0
I	0	0	0	0	0	0	0	0	2

degree matrix D

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

$$D = \text{diag}(5, 3, 4, 3, 2, 3, 3, 5, 2)$$

(b)

A	D	E	F	G
D	0	0	0	1
E	0	0	1	0
F	0	1	0	1
G	1	0	1	0

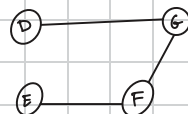
adj matrix A

D	D	E	F	G
D	1	0	0	0
E	0	1	0	0
F	0	0	2	0
G	0	0	0	2

deg matrix D

L	D	E	F	G
D	1	0	0	-1
E	0	1	-1	0
F	0	-1	2	-1
G	-1	0	-1	2

Laplacian matrix L



to find the eigenvalues (λ), we must solve the characteristic equation:

$$\det(L - \lambda I) = 0: \begin{vmatrix} 1-\lambda & 0 & 0 & -1 \\ 0 & 1-\lambda & -1 & 0 \\ 0 & -1 & 2-\lambda & -1 \\ -1 & 0 & -1 & 2-\lambda \end{vmatrix} = 0$$

we expand along the first row:

$$(1-\lambda) \begin{vmatrix} 1-\lambda & -1 & 0 \\ -1 & 2-\lambda & -1 \\ 0 & -1 & 2-\lambda \end{vmatrix} - (-1) \begin{vmatrix} 0 & 1-\lambda & -1 \\ 0 & -1 & 2-\lambda \\ -1 & 0 & -1 \end{vmatrix} = 0$$

$$\begin{aligned} (1-\lambda) \begin{vmatrix} 2-\lambda & -1 \\ -1 & 2-\lambda \end{vmatrix} - (-1) \begin{vmatrix} -1 & 0 \\ -1 & 2-\lambda \end{vmatrix} &= (1-\lambda) [(2-\lambda)^2 - 1] + 1 [-(2-\lambda) - 0] \\ &= (1-\lambda)(\lambda^2 - 4\lambda + 3) - (2-\lambda) = (-\lambda^3 + 5\lambda^2 - 7\lambda + 3) + \lambda - 2 \\ &= -\lambda^3 + 5\lambda^2 - 6\lambda + 1 \end{aligned}$$

$$(1-\lambda)(-\lambda^3 + 5\lambda^2 - 6\lambda + 1) = \lambda^4 - 6\lambda^3 + 11\lambda^2 - 7\lambda + 1$$

$$-1 \begin{vmatrix} 1-\lambda & -1 \\ -1 & 2-\lambda \end{vmatrix} = -1 [(1-\lambda)(2-\lambda) - 1] = -1 [\lambda^2 - 3\lambda + 1] = -\lambda^2 + 3\lambda - 1$$

$$(\lambda^4 - 6\lambda^3 + 11\lambda^2 - 7\lambda + 1) + (-\lambda^2 + 3\lambda - 1) = 0$$

$$\lambda^4 - 6\lambda^3 + 10\lambda^2 - 4\lambda = 0$$

$$\lambda(\lambda^3 - 6\lambda^2 + 10\lambda - 4) = 0 \quad (\lambda_1 = 0)$$

$$\lambda^3 - 6\lambda^2 + 10\lambda - 4 = 0$$

$$\text{testing } (\lambda = 2): \quad 2^3 - 6(2^2) + 10(2) - 4 = 8 - 24 + 20 - 4 = 0 \quad (\lambda_2 = 2)$$

$$\lambda^2 - 4\lambda + 2 = 0$$

$$\lambda = \frac{4 \pm \sqrt{(-4)^2 - 4 \cdot 1 \cdot 2}}{2} = \frac{4 \pm \sqrt{8}}{2} = 2 \pm \sqrt{2}$$

$\Rightarrow$ the four eigenvalues are: $\lambda_1 = 0$, $\lambda_2 = 2 - \sqrt{2}$, $\lambda_3 = 2$, $\lambda_4 = 2 + \sqrt{2}$

Let the eigenvectors $x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix}$

$$\begin{cases} (1 - \lambda) x_1 - x_4 = 0 \Rightarrow x_4 = (1 - \lambda) x_1 \\ (1 - \lambda) x_2 - x_3 = 0 \Rightarrow x_3 = (1 - \lambda) x_2 \\ -x_2 + (2 - \lambda) x_3 - x_4 = 0 \\ -x_1 - x_3 + (2 - \lambda) x_4 = 0 \end{cases}$$

$$\text{for } \lambda_1 = 0: \begin{cases} x_4 = x_1 \\ x_3 = x_2 \\ -x_2 + 2x_2 - x_1 = 0 \Rightarrow x_2 = x_1 \end{cases}$$

if we set $x_1 = 1$ then we have the eigenvector 1: $v_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}$

$$\text{for } \lambda_2 = 2: \begin{cases} x_4 = -x_1 \\ x_3 = -x_2 \\ -x_2 + 0 - x_4 = 0 \Rightarrow x_4 = -x_2 \end{cases}$$

if we set $x_1 = 1$ then we have the eigenvector 2: $v_2 = \begin{pmatrix} 1 \\ -1 \\ -1 \\ 1 \end{pmatrix}$

- for $\lambda_3 = 2 - \sqrt{2}$:

$$\begin{cases} x_4 = (\sqrt{2} - 1)x_1 \\ x_3 = (\sqrt{2} - 1)x_2 \\ -x_2 + \sqrt{2}(\sqrt{2} - 1)x_2 - (\sqrt{2} - 1)x_1 = 0 \Rightarrow x_2 = -x_1 \end{cases}$$

if we set $x_1 = 1$ then we have the eigenvector 3: $v_3 = \begin{pmatrix} 1 \\ -1 \\ 1 - \sqrt{2} \\ \sqrt{2} - 1 \end{pmatrix}$

- for $\lambda_4 = 2 + \sqrt{2}$:

$$\begin{cases} x_4 = (-1 - \sqrt{2})x_1 \\ x_3 = (-1 - \sqrt{2})x_2 \\ -x_2 - \sqrt{2}(-1 - \sqrt{2})x_2 - (-1 - \sqrt{2})x_1 = 0 \Rightarrow x_2 = -x_1 \end{cases}$$

if we set $x_1 = 1$ then we have the eigenvector 4: $v_4 = \begin{pmatrix} 1 \\ -1 \\ 1 + \sqrt{2} \\ 1 - \sqrt{2} \end{pmatrix}$

(c) look at the code file.

(d)

```
Laplacian Matrix (L):
[[ 5 -1 -1 -1 0 0 0 -1 -1]
 [-1 3 -1 0 0 0 0 -1 0]
 [-1 -1 4 -1 -1 0 0 0 0]
 [-1 0 -1 3 0 0 -1 0 0]
 [ 0 0 -1 0 2 -1 0 0 0]
 [ 0 0 0 0 -1 3 -1 -1 0]
 [ 0 0 0 -1 0 -1 3 -1 0]
 [-1 -1 0 0 0 -1 -1 5 -1]
 [-1 0 0 0 0 0 0 -1 2]]

Eigenvalues (Sorted):
[-0.      1.3305  1.9428  2.      2.7816  4.4379  5.268   5.6878  6.5513]

Eigenvectors:
[[-0.3333  0.2387  0.1789 -0.      0.0205 -0.1668 -0.0348 -0.6055 -0.6355]
 [-0.3333  0.1895  0.414  -0.     -0.6428  0.1562 -0.2709  0.4068 -0.072 ]
 [-0.3333 -0.0655  0.4569 -0.     0.1046 -0.0796  0.7808  0.0464  0.2162]
 [-0.3333  0.0383  0.2112 -0.5    0.5356 -0.2517 -0.4199  0.1885  0.1754]
 [-0.3333 -0.6413  0.1358  0.5    0.2141  0.2971 -0.2646 -0.068  -0.0196]
 [-0.3333 -0.3638 -0.4492  0.     -0.2719 -0.6448  0.0838  0.2045 -0.1272]
 [-0.3333 -0.1092 -0.4125 -0.5    -0.0081  0.6083  0.2062  0.0526 -0.2036]
 [-0.3333  0.1432 -0.1982  0.     -0.2655  0.0218 -0.1317 -0.5343  0.6748]
 [-0.3333  0.5704 -0.3371  0.5     0.3134  0.0595  0.0509  0.3091 -0.0086]]
```

The Laplacian matrix (L) of our connected graph produces a first eigenvalue of 0, which is expected. To identify the community structure

of the graph, we analyze the eigenvector corresponding to the second smallest eigenvalue ($\lambda = 1,3305$).

By extracting this second eigenvector and applying a threshold of $s = 0$, we can partition the graph into two distinct communities based on the sign of their vector components:

- Community 1 (Positive Values): Nodes A, B, D, H and I.

- Community 2 (Negative Values): Nodes C, E, F and G.

Nodes C and D have values very close to zero, indicating they sit on the boundary between two communities. This mathematical partition successfully identifies the two most densely connected clusters in the network while minimizing the number of cuts between them.

(e) We need to find a tightly coupled pair of nodes with the fewest number of outgoing arrows to the rest of the graph.

We have the pairs: $\{A, I\}$, $\{D, G\}$, $\{E, F\}$, $\{A, H\}$, $\{F, G\}$ and $\{G, H\}$

- for the pair $\{A, I\}$ we have to remove 4 edges
- for the pair $\{D, G\}$ we have to remove 3 edges.
- for the pair $\{E, F\}$ we need to remove 2 edges.
- for the pair $\{A, H\}$ we need to remove 6 edges.

- for the pair $\{F, G\}$ we need to remove 3 edges.
- for the pair $\{G, H\}$ we need to remove 5 edges.

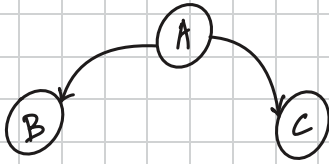
⇒ The pair $\{E, F\}$ is our optimal candidate because it only has two external out links.

By removing those two arrows, the only out link Node E has left is $E \rightarrow F$, and the only out link Node F has left is $F \rightarrow E$. This isolates nodes E and F into a perfectly closed loop. Because both E and F still have an out degree of exactly 1, neither of them becomes a dead-end.

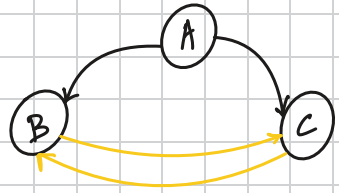
(†) It's impossible to create a dead-end by adding edges. A dead-end is defined as a node with an out degree of exactly zero. Adding an outgoing edge to a node increases its out degree, and adding an incoming edge does not affect its out degree. Therefore, adding edges can never reduce node's out degree to zero.

On the other hand, it is possible to create a spider trap by adding edges, provided the connected graph initially contains dead ends. By adding directed edges that connect existing dead ends to one another, we give them an out degree greater than zero (satisfying the condition that they are no longer dead ends) while ensuring all out links remain strictly within

that newly formed group (creating the spider trap).



after adding two
directed edges



Task 2:

(a) The PCY (Park-Chen-Yu) algorithm finds frequent item sets without crashing the computer memory like the "Apriori" method, and it works in three phases:

- Phase 1: It counts individual items, and using leftover memory, it also pairs up items in each basket, hashes them into numbered buckets and just tracks the bucket count (without saving the actual pairs).
- Phase 2: If a bucket total count reaches the threshold, it gets a "1" in its bitmap value, if not it gets a "0".
- Phase 3: To be officially counted, a pair must pass two tests: both items must be frequent and it must hash to a bucket with a "1".

So, by using extra calculations (CPU power) to create this filter, PCY avoids saving millions of (non frequent) pairs in memory, preventing the RAM crashes that happen with "Apriori" method.

(b)

We have five items:

1 = "Toyota" 2 = "Honda" 3 = "Ford" 4 = "BMW" 5 = "Tesla"

and we have six baskets:

Basket 1 = {Toyota, Honda, Ford} = {1, 2, 3}

Basket 2 = {Toyota, Honda, BMW} = {1, 2, 4}

Basket 3 = {Honda, Ford, BMW} = {2, 3, 4}

Basket 4 = {Toyota, Honda, Ford} = {1, 2, 3}

Basket 5 = {Ford, BMW, Tesla} = {3, 4, 5}

Basket 6 = {Toyota, Ford, Tesla} = {1, 3, 5}

So, the support of each item is:

- {Toyota} = 4
- {Honda} = 4
- {Ford} = 5
- {BMW} = 3
- {Tesla} = 2

Since our threshold $s=3$, so the item {Tesla} does not meet it.

Hence, our frequent items are: {Toyota}, {Honda}, {Ford} and {BMW}.

(c) A pair cannot possibly appear more times than its individual items, meaning if a pair meets the support threshold, both of the pair items must meet or exceed that threshold.

(d) Basket 1 = $\{1, 2, 3\}$ has three possible pairs:

- $\{1, 2\} \rightarrow (1 \times 2) \bmod 7 = 2$
- $\{1, 3\} \rightarrow (1 \times 3) \bmod 7 = 3$
- $\{2, 3\} \rightarrow (2 \times 3) \bmod 7 = 6$

} our buckets

Basket 2 = $\{1, 2, 4\}$ has three possible pairs:

- $\{1, 2\} \rightarrow (1 \times 2) \bmod 7 = 2$
- $\{1, 4\} \rightarrow (1 \times 4) \bmod 7 = 4$
- $\{2, 4\} \rightarrow (2 \times 4) \bmod 7 = 1$

Basket 3 = $\{2, 3, 4\}$ has three possible pairs:

- $\{2, 3\} \rightarrow (2 \times 3) \bmod 7 = 6$
- $\{2, 4\} \rightarrow (2 \times 4) \bmod 7 = 1$
- $\{3, 4\} \rightarrow (3 \times 4) \bmod 7 = 5$

Basket 4 = $\{1, 2, 3\}$ has three possible pairs (exactly like basket 1):

- $\{1, 2\} \rightarrow (1 \times 2) \bmod 7 = 2$
- $\{1, 3\} \rightarrow (1 \times 3) \bmod 7 = 3$
- $\{2, 3\} \rightarrow (2 \times 3) \bmod 7 = 6$

Basket 5 = $\{3, 4, 5\}$ has three possible pairs:

- $\{3, 4\} \rightarrow (3 \times 4) \bmod 7 = 5$
- $\{3, 5\} \rightarrow (3 \times 5) \bmod 7 = 1$
- $\{4, 5\} \rightarrow (4 \times 5) \bmod 7 = 6$

Basket 6 = $\{1, 3, 5\}$ has three possible pairs:

- $\{1, 3\} \rightarrow (1 \times 3) \bmod 7 = 3$
- $\{1, 5\} \rightarrow (1 \times 5) \bmod 7 = 5$
- $\{3, 5\} \rightarrow (3 \times 5) \bmod 7 = 1$

Now, let's build our bucket table:

Bucket	Pairs	Count
0	None	0
1	$\{2,4\}, \{1,4\}, \{3,5\}, \{3,5\}$	4
2	$\{1,2\}, \{4,2\}, \{1,2\}$	3
3	$\{1,3\}, \{1,3\}, \{1,3\}$	3
4	$\{1,4\}$	1
5	$\{3,4\}, \{3,4\}, \{1,5\}$	3
6	$\{2,3\}, \{2,3\}, \{2,3\}, \{4,5\}$	4

(e) As we can see in our table, only the buckets that have a count value equal or bigger than our threshold $s=3$ will be marked as frequent buckets (which means they will have a bitmap value of "1"). Those buckets are: Bucket 1, 2, 3, 5 and 6. (we don't mark bucket 4 because $1 < 3$).

(f) If a pair is truly frequent, it will appear in the dataset at least s times. Therefore, it will add a count to its target bucket at least s times, guaranteeing that the bucket's total count reaches the threshold s to become a frequent bucket. Otherwise, if a bucket total count is less than the threshold s , it is mathematically impossible for any single pair inside that bucket to have appeared s times. Hence, we can conclude that any pair landing in a non-frequent bucket is not a frequent pair.

(g) A bucket gets its total count by summing the occurrences of all pairs that land in it. For example, the different pairs $\{2,4\}$ and $\{3,5\}$ both hash to bucket 1. Pair $\{2,4\}$ appears 2 times and pair $\{3,5\}$ appears 2 times. Summing them gives a total count of 4, which is greater than the threshold. This makes this bucket frequent, even though the two individual pairs inside it are not frequent on their own. Because of this hash collision, a frequent bucket does not guarantee its pairs are frequent.

- (h) For a pair to be counted on "Pass 2", it must satisfy 2 conditions:
- * Both individual items in the pair must be frequent.
 - * The pair must hash to a frequent bucket.

(i) The frequent items that we have: $\{1, 2, 3, 4\}$

So, we would have these pairs:

- $\{1, 2\}$: Both items are frequent; hashes to bucket 2
- $\{1, 3\}$: Both items are frequent; hashes to bucket 3
- $\{1, 4\}$: Both items are frequent; hashes to bucket 4
- $\{2, 3\}$: Both items are frequent; hashes to bucket 6
- $\{2, 4\}$: Both items are frequent; hashes to bucket 1
- $\{3, 4\}$: Both items are frequent; hashes to bucket 5

So we would not consider the pair $\{1, 4\}$ because it hashes to a non frequent bucket.

If we used the "A priori" method with the 4 frequent items $\{1, 2, 3, 4\}$, we would have to count all 6 possible pairs (with saving them) even though the pair $\{1, 4\}$ will end up non frequent.

(j) The surviving pairs are $\{1, 2\}$, $\{1, 3\}$, $\{2, 3\}$, $\{1, 4\}$ and $\{3, 4\}$

Let's count their frequency:

- $\{1, 2\} = 3 \rightarrow$ appeared in baskets 1, 2, 4.
- $\{1, 3\} = 3 \rightarrow$ appeared in baskets 1, 4, 6.
- $\{2, 3\} = 3 \rightarrow$ appeared in baskets 1, 3, 4
- $\{1, 4\} = 2 \rightarrow$ appeared in baskets 2, 3
- $\{3, 4\} = 2 \rightarrow$ appeared in baskets 3, 5

So yes, there are survivors that are not frequent, both pair $\{2, 4\}$ and $\{3, 4\}$ survived the PCY filter (because they hashed to frequent buckets), but their actual count was just 2. These are the "false positive" that the hash filter let slip through.

(K) The whole goal of the algorithm is to find All frequent pairs. If the filter accidentally throws away a truly frequent pair, that data is lost forever. We don't mind letting a few fake ones slip through (False positives), because we double check them in pass 2, but we can't afford to lose real frequent pairs.

It is ensured by "addition" rule in question "g" that a bucket's total count is made by adding up the counts of every pair inside it. Because of this, a bucket's total score will always be at least as high as any single pair inside it. If a pair appears 3 times, it gives the bucket 3 points. Therefore, the bucket is guaranteed to reach the threshold of 3, making it impossible for the true pair to be discarded.

- (L) • Using very few buckets: The filter becomes useless, too many different pairs get into the same buckets, their count accidentally add up to pass the threshold, letting a lot of fake pairs slip through the filter.
- Using very many buckets: The filter works great, pairs are spread out, so the "non frequent" ones don't mix together and stay under the threshold and fail the test, giving way less "false positives".
- The memory cost of very many buckets: Every single bucket needs a little bit of computer memory (RAM) to hold its score, if we create too many buckets, just holding them will eat up all the RAM and crash the system, which defeats the whole purpose of using PCY in the first place.